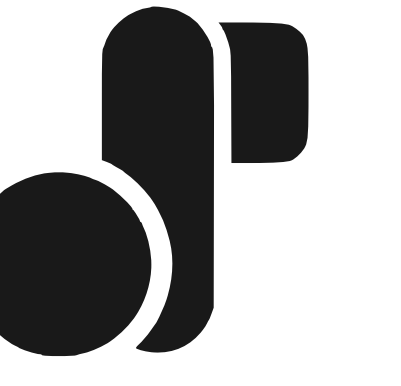




Flowly Use-Case Sunumu

Hakan Ören
Berke Toprak

Temmuz 2026



İçindekiler

1. Flowly Nedir?
2. Temel Kavramlar
3. Sistem Mimarisi
4. Bir İsteğin Başlangıçtan Sonuca Yolculuğu
5. Detaylı Kullanım Senaryoları (Use Case'ler)
 - a. Problem
 - b. Kullanıcının İsteği
 - c. Agent'ın İzlediği Süreç
 - d. Kullanılan Araçlar
 - e. Hafıza ile Etkileşim
 - f. Üretilen Çıktı
 - g. Sağlanan Fayda
6. Güvenlik ve Yetkilendirme Modeli
7. Genişletilebilirlik
8. Flowly'yi Farklı Kılan Teknik Özellikler
9. Ekler

1. Flowly Nedir?

Yapay zeka son birkaç yılda büyük bir sıçrama yaşadı. Bugün büyük dil modelleri metin yazabiliyor, kod üretebiliyor, belgeleri analiz edebiliyor ve karmaşık problemler üzerine akıl yürütebiliyor. Ancak bu sistemlerin büyük çoğunluğu hâlâ tek bir temel probleme sahip: **Konuşabiliyorlar ama çalışamıyorlar.**

Kullanıcılar gerçek bir işi tamamlamak istediğinde hâlâ onlarca farklı uygulama arasında geçiş yapmak zorunda kalıyor. E-postalarını kontrol ediyor, takvimine bakıyor, tarayıcıda araştırma yapıyor, belgelerini açıyor, kod deposunu inceliyor ve bütün bu süreci yapay zekaya sürekli yeniden anlatıyor. Her yeni sohbet önceki bağlamın büyük kısmını kaybediyor; tekrar eden görevler yeniden tarif ediliyor ve üretilen çıktılar çoğu zaman yalnızca sohbet geçmişinde kalan geçici cevaplara dönüşüyor.

Flowly bu problemi çözmek amacıyla geliştirildi. Flowly, yalnızca cevap üreten bir sohbet botu olmakla kalmıyor kullanıcının kendi bilgisayarında çalışan, araçları kullanabilen, uzun dönem hafızaya sahip olan ve kendisine verilen görevleri baştan sona tamamlayabilen bir **agent**'dir. Bir soruya cevap vermek yerine, problemi analiz eder, gerekli araçları belirler, uygun modeli kullanır, güvenlik protokollerini uygular ve görevi sonuçlandırır.

Flowly'nin temel yaklaşımı, büyük dil modelini sistemin merkezi yapmak yerine onu yalnızca karar veren bileşenlerden biri olarak kullanmaktır. Hafıza yönetimi, araç yürütme, izin sistemi, görev planlama, çok adımlı iş akışları ve durum yönetimi tamamen Flowly tarafından kontrol edilir. Böylece kullanılan model değişse bile sistemin davranışı ve çalışma prensibi aynı kalır.

Bu yaklaşım sayesinde kullanıcı farklı cihazlardan Flowly'ye erişse bile aslında her zaman aynı agent ile konuşur. Masaüstü uygulaması, mobil uygulama, terminal veya mesajlaşma platformları yalnızca farklı erişim noktalarıdır. Hafıza tektir, kimlik tektir ve yürütme ortamı tektir. Kullanıcının geçmiş projeleri, çalışma alışkanlıkları ve izinleri tek bir sistem içerisinde korunur ve zamanla gelişmeye devam eder. Üstelik hem mobil hem de web platformlarının tamamından kesintisiz olarak erişim mümkündür. İş akışı tam zamanlı izlenir, tam zamanlı yönetilir. Tüm bunlarla birlikte agent bağlamsal hafızası sayesinde zamanla iş yapış şeklini daha iyi öğrenir, daha etkili sonuçlar üretir ve kullanıcıyı daha iyi anlayabilecek seviyeye gelir.

2. Temel Kavramlar

Flowly, birbirleriyle etkileşim halinde çalışan farklı bileşenlerden oluşur. Bu bölümde dokümanın devamında kullanılacak temel kavramlar özetlenmiştir.

Agent: Kullanıcının isteğini analiz eden, görevleri planlayan ve gerekli araçları kullanarak süreci yöneten ana yürütme bileşenidir.

Memory: Kullanıcının önemli bilgilerini uzun vadeli olarak saklayan ve gerektiğinde Agent'a bağlam sağlayan kalıcı hafıza sistemidir.

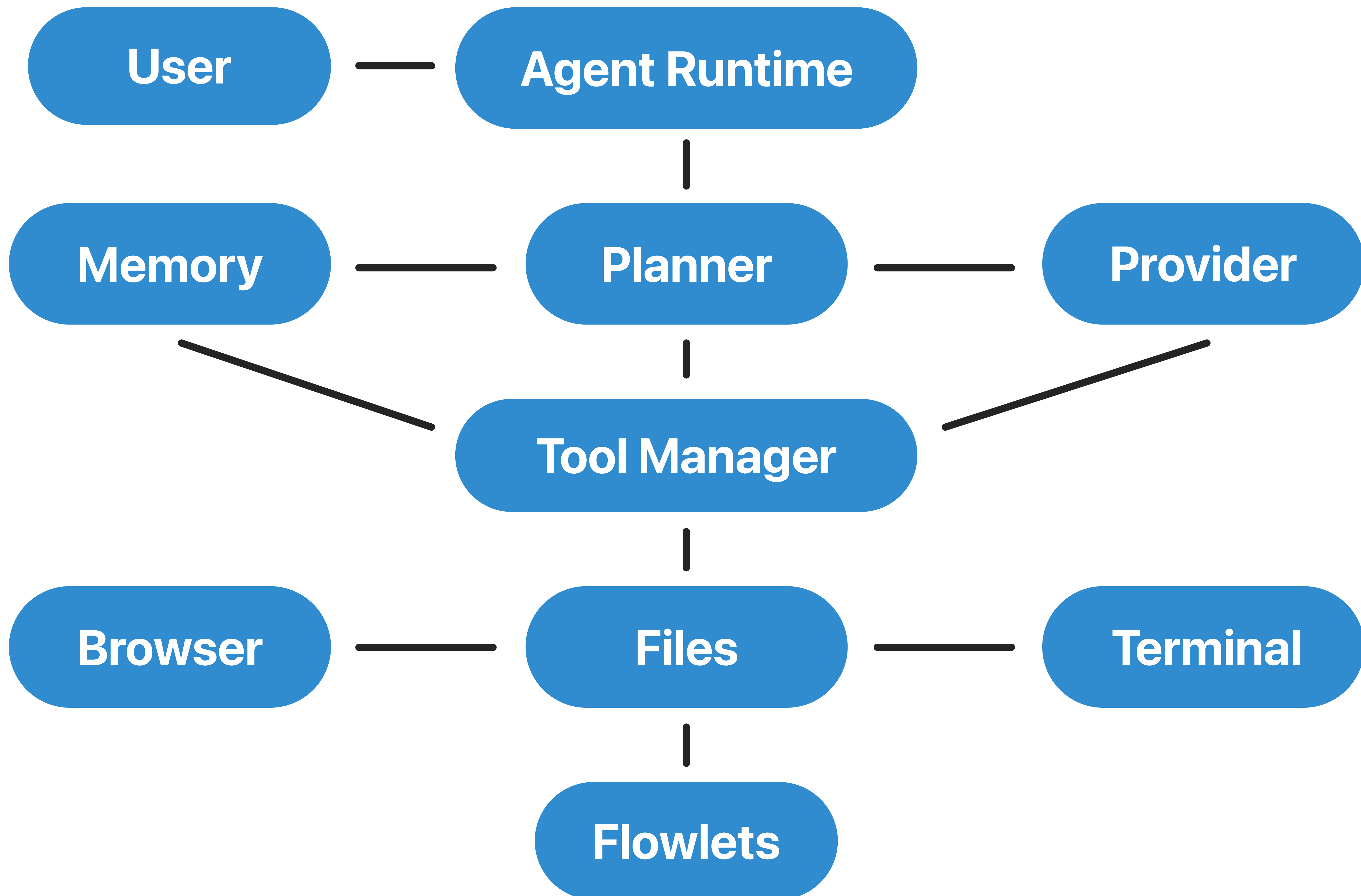
Tool: Agent'ın dosya sistemi, internet, API'ler, terminal ve diğer uygulamalarla etkileşim kurmasını sağlayan yürütülebilir yeteneklerdir.

Skill: Agent'ın belirli bir konuda uzmanlaşmasını sağlayan, önceden tanımlanmış davranış ve yetenek paketleridir.

Flowlets: Doğal dil ile oluşturulan, belirli görevleri kolaylaştıran küçük ve etkileşimli kullanıcı arayüzleridir.

Provider: Flowly'nin kullandığı büyük dil modelini sağlayan katmandır. Sistem farklı model sağlayıcılarıyla çalışabilecek şekilde tasarlanmıştır.

Runtime: Agent, Memory, Tool ve diğer bileşenlerin birlikte çalışmasını sağlayan temel çalışma ortamıdır.



Prompt İşleyiş Diagramı

3. Sistem Mimarisi

Flowly, farklı erişim kanallarını tek bir çalışma ortamında birleştiren katmanlı bir mimariye sahiptir. Kullanıcı; terminal, masaüstü ve mobil uygulamalar veya mesajlaşma kanalları üzerinden sisteme ulaşabilir. Kullanılan kanal değişse bile aynı Agent, hafıza ve yetenekler kullanılmaya devam eder.

Kullanıcı Arayüzleri ve Kanallar: Kullanıcının Flowly ile iletişim kurduğu arayüzlerdir. Terminal (CLI), masaüstü, mobil ve mesajlaşma uygulamalarından gelen istekler ortak bir iletişim katmanında karşılanır.

Flowly Gateway: Tüm kullanıcı tarafından gönderilen promptların sisteme giriş yaptığı merkezdir. Gelen mesajı doğru kullanıcıya ve oturuma yönlendirir; işlem sonucunu yeniden kullanıcının bulunduğu kanala (örn: Telegram veya Flowly iOS uygulaması) iletir.

Agent Runtime: Sistemin ana yürütme katmanıdır. Kullanıcının isteğini analiz eder, yapılması gereken adımları planlar ve uygun bileşenleri kullanarak görevi tamamlar. Çok adımlı işlerin ve alt görevlerin koordinasyonu bu katmanda gerçekleştirilir.

Hafıza, Skills ve Tools: Agent, geçmiş konuşmaları ve kalıcı bilgileri hafıza katmanından alır. Önceden tanımlanmış yetenekleri Skills üzerinden, dosya sistemi, internet, terminal ve diğer uygulamalarla etkileşimini ise Tools üzerinden yürütür.

Model Sağlayıcı (Provider): Agent'ın metni anlaması, karar vermesi ve içerik üretmesi için gerekli büyük dil modelini sağlar. Flowly farklı model sağlayıcılarıyla veya yerel modellerle çalışabilir; sağlayıcı değişse bile sistemin geri kalan mimarisi aynı kalır.

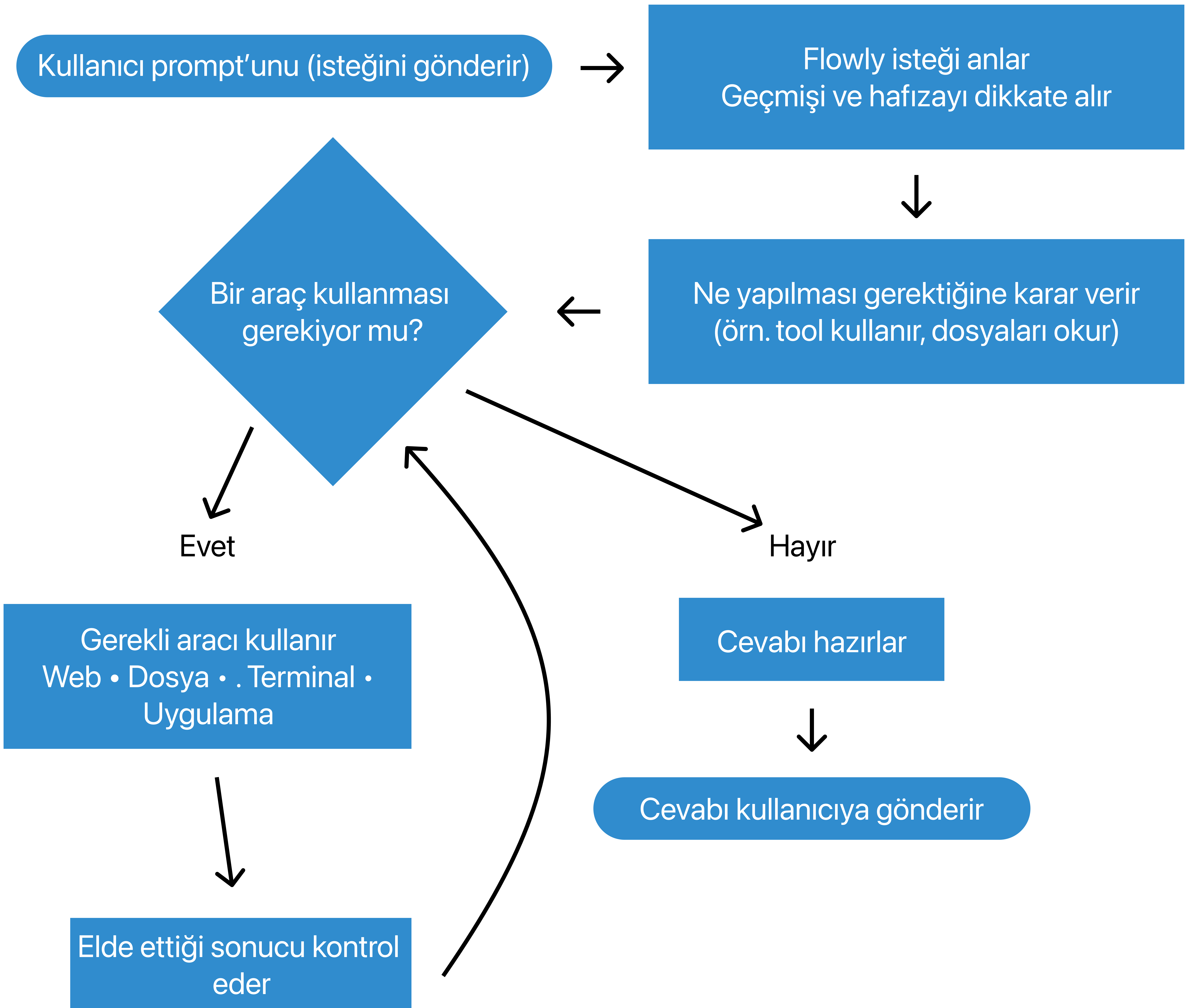
Tüm bu süreç kullanıcının kontrolündeki çalışma ortamında yürütülür. Dosya ve terminal gibi kritik işlemler güvenli çalışma alanı, izin politikaları ve kullanıcı onaylarıyla sınırlandırılır. Böylece Flowly, farklı kanallardan erişilebilen ancak merkezi olarak yönetilen, güvenli ve genişletilebilir bir Agent sistemi sunar.



4. Bir İsteğin Başlangıçtan Sonuca Yolculuğu

Flowly'de bir kullanıcı isteği doğrudan yapay zekâ modeline gönderilmez. İstek önce geldiği kanal tarafından ortak mesaj biçimine dönüştürülür ve ilgili oturumla eşleştirilir. Ardından konuşma geçmişi, sistem prompt'u, persona ve ilgili hafıza bilgileri kullanılarak çalışma bağlamı hazırlanır.

Agent bu bağlamı model sağlayıcıya iletir. Model doğrudan cevap verebilir veya bir araç kullanılmasını isteyebilir. Araç gerektiğinde güvenlik ve onay kontrolleri uygulanır, araç çalıştırılır ve elde edilen sonuç tekrar modele gönderilir. Bu döngü görev tamamlanana kadar devam eder. Sonuç, oturum geçmişi ve işlem kayıtları kaydedildikten sonra kullanıcının isteği gönderdiği kanala veya uygulamaya geri iletilir.



5. Detaylı Kullanım Senaryoları (Use Case'ler)

Senaryo 1 - Tek Cümleyle Kişisel Bir Uygulama Oluşturma

Problem: Kullanıcı harcamalarını takip etmek istemektedir ancak yeni bir uygulama kurmak, tablolar hazırlamak ve her fişi elle girmek istememektedir.

Kullanıcının İsteği:

"Bana bir harcama ekranı oluştur. Fişin fotoğrafını çektiğimde mağaza, tutar ve kategoriyi çıkarsın. Aylık toplamı ve kategori grafiğini gösterebilirsin. Bütçemin yüzde 80'ine ulaştığımda beni uyarısın."

Agent'in İzlediği Süreç:

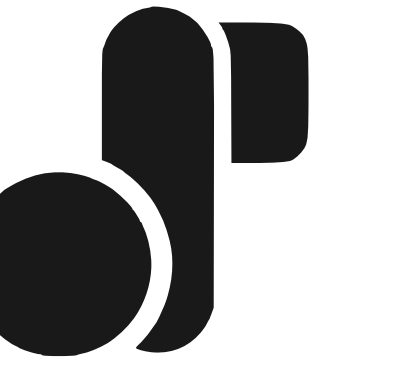
1. Flowly, kullanıcının tarifinden çalışan bir Flowlet oluşturur.
2. Harcama listesi, aylık toplam, bütçe durumu ve kategori grafiğini hazırlar.
3. Kullanıcı fiş fotoğrafı eklediğinde izole bir görüntü okuma işlemiyle fişteki bilgileri çıkarır.
4. Bulunan değerleri düzenlenebilir bir harcama kaydı olarak ekler.
5. Toplamları ve grafikleri her yeni kayıta otomatik olarak günceller.
6. Bütçe belirlenen seviyeye ulaştığında Flowlet'in kendi uyarı kuralı çalışır.

Kullanılan Flowly Özellikleri: Flowlets, fotoğraftan veri çıkarma, canlı hesaplamalar, grafikler ve koşula bağlı bildirimler.

Hafıza ile Etkileşim: Harcama kayıtları uzun dönem hafızaya yazılmaz. Flowlet'e ait ayrı ve yerel veritabanında saklanır. Kullanıcı daha sonra "Bu ay markete ne kadar harcadım?" diye sorduğunda Agent canlı Flowlet verisini okuyabilir.

Üretilen Çıktı: Masaüstü ve iPhone'da çalışan, kalıcı ve etkileşimli kişisel harcama uygulaması. Sağlanan Fayda: Flowly yalnızca tavsiye veya rapor üretmez; kullanıcının tarifinden, kod yazmadan kullanabileceği gerçek bir mini uygulama oluşturur. Ekrandaki normal buton ve hesaplamalar her kullanımda yapay zekâ çağırmadan anında çalışır.

Ön Koşul: Flowlet destekleyen Flowly masaüstü veya mobil uygulamasının kullanılması gerekir.



Senaryo 2 - Toplantı Sırasında Sessiz Bir Danışman

Problem: Kullanıcı önemli bir satış veya müşteri görüşmesi sırasında itirazları, verilen sözleri ve sorulması gereken kritik soruları kaçırabilmektedir.

Kullanıcının İsteği:

"Bu görüşmede beni yönlendir. Sadece gerçekten önemli bir şeyi kaçırsam uyar. Müşteri bir itiraz dile getirirse nasıl cevap verebileceğimi söyle. Toplantı bitince kararları ve sonraki adımları çıkar."

Agent'in İzlediği Süreç:

1. Flowly masaüstü uygulaması görüşmeyi canlı olarak metne çevirir.
2. Flowly son konuşma bölümlerini sürekli olarak değerlendirir.
3. Her cümlede kullanıcıyı bölmek yerine, önce önerinin gerçekten gerekli olup olmadığını kontrol eder.
4. Kritik bir an tespit edildiğinde kullanıcının ekranında kısa ve özel bir öneri gösterir.
5. Kullanıcı isterse "Şimdi öneri ver" seçeneğiyle anlık yardım alabilir.
6. Görüşme durdurulduğunda toplantıyı özetler, kararları ve önemli kişi/proje bilgilerini çıkarır.
7. Görüşme metnini daha sonra açılacak yerel bir doküman olarak saklar.

Kullanılan Flowly Özellikleri: Meeting Coach, canlı transkript değerlendirmesi, öneri filtresi, Knowledge Graph, yönetilen hafıza ve Artifacts.

Hafıza ile Etkileşim: Toplantıdaki kalıcı kararlar ve ilişkiler bilgi grafiğine veya yönetilen hafızaya alınabilir. Belirsiz ya da hassas çıkarımlar doğrudan doğru kabul edilmek yerine kullanıcı incelemesine bırakılabilir.

Üretilen Çıktı: Toplantı sırasında zamanında öneriler; toplantı sonunda özet, kararlar, sonraki adımlar ve kayıtlı transkript.

Sağlanan Fayda: Flowly toplantı bittikten sonra yalnızca not çıkarmakla kalmaz; görüşme devam ederken kullanıcının yanında çalışan özel bir danışmana dönüşür.

Ön Koşul: Canlı transkript gönderebilen Flowly masaüstü uygulamasında Coaching oturumunun başlatılması gerekir. Flowly bu akışta doğrudan ham sesi değil, uygulamanın oluşturduğu konuşma metnini işler.

Senaryo 3 - Tekrarlanan İşi Fark Edip Kendi Yeteneğini Yazma

Problem: Kullanıcı her hafta aynı işi istemekte ancak raporun biçimini, kontrol kurallarını ve izlenecek adımları her seferinde yeniden anlatmaktadır.

Tekrarlanan Kullanıcı İsteği:

"Bu haftaki satış dosyasını bölgelere göre özetle, geçen haftayla karşılaştı, yüzde 20'den büyük değişimleri işaretle ve aynı formatta rapor hazırla."

Agent'in İzlediği Süreç:

1. Flowly, aynı çalışma biçiminin farklı görüşmelerde tekrarlandığını tespit eder.
2. Varsayılan eşik olarak isteğin en az üç kez ve en az iki ayrı oturumda görülmesini bekler.
3. Tekrarlanan adımlardan yeniden kullanılabilir bir Skill taslağı oluşturur.
4. Skill'i yazmadan önce mevcut Skill klasörünün geri alınabilir yedeğini oluşturur.
5. Hazırlanan Skill'i doğrular, işlem kaydına ekler ve Agent tarafından oluşturulduğunu işaretler.
6. Sonraki benzer istekte yeni Skill'i kullanarak aynı kontrol sırasını ve rapor biçimini uygular.
7. Kullanıcı sonucu beğenmezse tek işlemi geri alabilir veya tüm Skill klasörünü önceki sürüme döndürebilir.

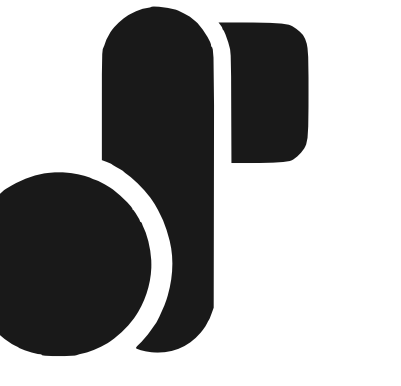
Kullanılan Flowly Özellikleri: Oturumlar arası tekrar tespiti, Skill mining, Skill oluşturma, kullanım takibi, snapshot, işlem günlüğü ve rollback.

Hafıza ile Etkileşim: Burada kişisel bir bilgi değil, tekrar eden bir çalışma yöntemi öğrenilir. Yani Flowly yalnızca "kullanıcı neyi seviyor?" sorusunu değil, "bu işi kullanıcı için nasıl yapıyorum?" sorusunu da hatırlar.

Üretilen Çıktı: Kullanıcıya özel, tekrar kullanılabilir ve geri alınabilir yeni bir Flowly Skill'i.

Sağlanan Fayda: Kullanıcı Flowly'yi elle programlamak zorunda kalmaz. Sistem gerçek kullanım geçmişinden kendi çalışma yöntemini geliştirir ve zamanla kullanıcının iş yapış biçimine uyum sağlar.

Ön Koşul: Skill self-improvement özelliği kullanıcı tarafından etkinleştirilmelidir. Özellik varsayılan olarak kapalıdır ve yalnızca Agent'in kendi oluşturduğu Skill'leri yönetir; kullanıcının veya sistemin kurulu Skill'lerine dokunmaz.



Senaryo 4 - Agent'ın Kullanıcıyı Gerçekten Telefonla Araması

Problem: Kullanıcı bazı hatırlatmaları mesaj olarak gördüğünde ertelemekte veya yoğunluk içinde tamamen kaçırmaktadır.

Kullanıcının İsteği:

"Yarın sabah 09.00'da beni telefonla ara. Günün üç önceliğini birlikte belirleyelim. Bana sırayla sorular sor ve görüşmenin özetini Telegram'a gönder."

Agent'ın İzlediği Süreç:

1. Flowly belirtilen saat için tek seferlik bir telefon araması planlar.
2. Saat geldiğinde kullanıcının gerçek telefon numarasını arar.
3. Kullanıcının konuşmasını metne çevirir ve cevaplarını sesli olarak iletir.
4. Öncelikler ve engeller hakkında karşılıklı bir görüşme yürütür.
5. Görüşme sona erdiğinde konuşmanın kısa bir özetini hazırlar.
6. Telefon görüşmesi Telegram hesabıyla bağlıysa özeti aynı Telegram konuşmasına gönderir ve oturum geçmişine ekler.

Kullanılan Flowly Özellikleri: Cron, gerçek telefon araması, konuşmayı metne çevirme, ses üretme ve kanallar arası oturum bağlantısı.

Hafıza ile Etkileşim: Görüşme, bağlı olduğu Flowly oturumuyla ilişkilendirilebilir. Böylece kullanıcı daha sonra Telegram'dan "Sabah konuştuğumuz ikinci öncelik neydi?" diye sorabilir.

Üretilen Çıktı: Gerçek bir telefon görüşmesi ve görüşme sonrasında Telegram'a gönderilen özet.

Sağlanan Fayda: Kullanıcının Agent'a gitmesini beklemek yerine Agent gerektiği anda kullanıcıya ulaşır. Telefon görüşmesi, sohbet geçmişi ve mesajlaşma kanalı tek bir devamlı süreç hâline gelir.

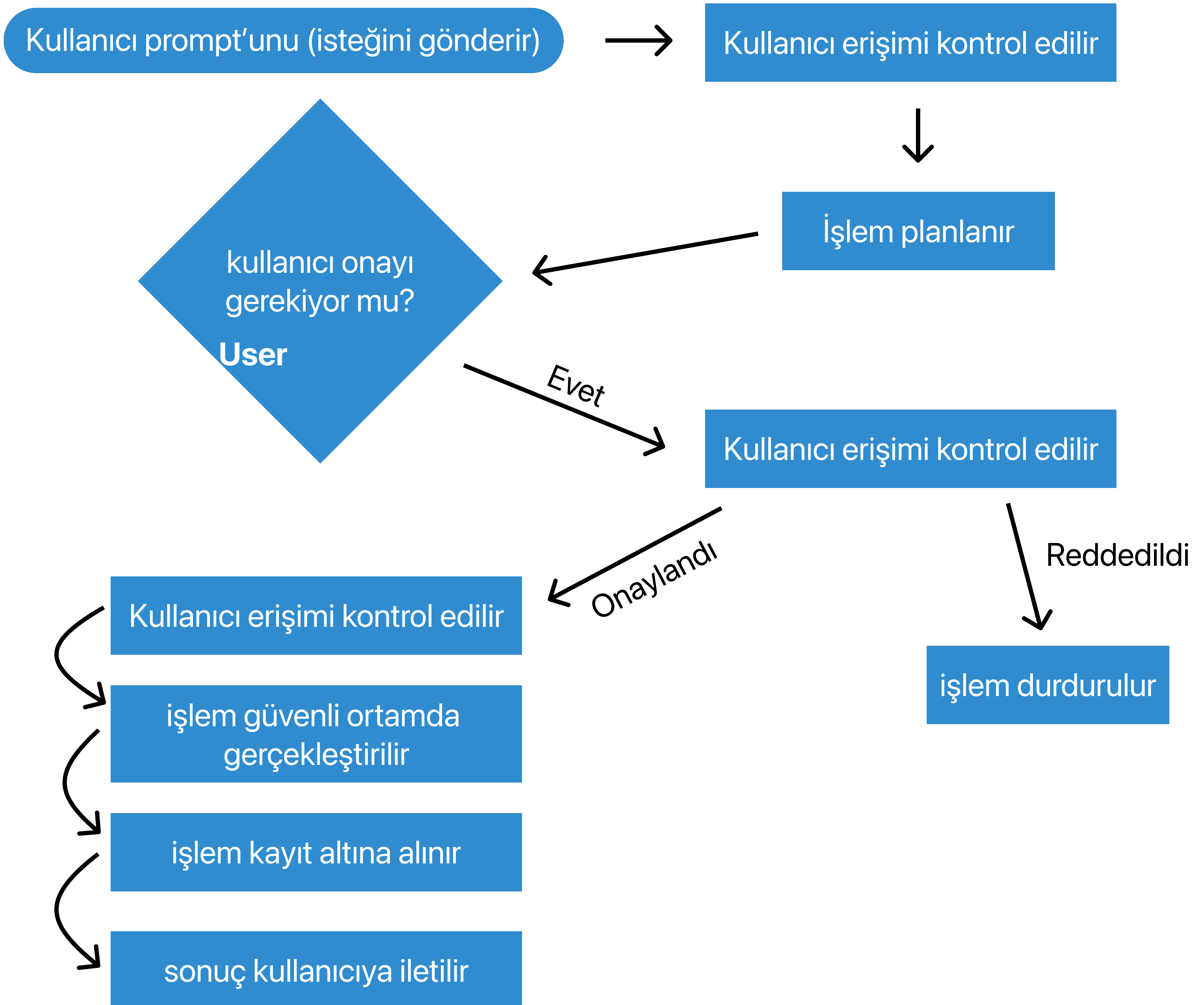
Ön Koşul: Twilio, herkese açık güvenli webhook adresi, desteklenen konuşmayı metne çevirme servisi ve ses üretme servisi yapılandırılmalıdır. Arama zamanında Flowly Gateway çalışıyor olmalıdır.

6. Güvenlik ve Yetkilendirme Modeli

Flowly'nin güvenlik modeli, yapay zekâya sınırsız yetki vermek yerine işlemleri farklı kontrol katmanlarından geçirir. Model yapılacak işlemi belirler; Flowly kullanıcının erişim yetkisini, işlem izinlerini ve korunması gereken alanları kontrol eder.

Komut çalıştırma, dosya değiştirme veya harici bir serviste işlem yapma gibi adımlar kullanıcı onayına bağlanabilir. SSH anahtarları, tarayıcı oturumları ve servis kimlik bilgileri gibi hassas alanlara erişim ayrıca engellenir. Gerçekleştirilen işlemler, hassas bilgiler gizlenerek cihaz üzerinde kayıt altına alınır.

Flowly'nin temel yaklaşımı şöyledir: Model işlemi önerir, kullanıcı yetki verir, sistem güvenlik sınırlarını uygular. İşletim sistemi son güvenlik sınırıdır; bu nedenle yalnızca güvenilir eklentiler kullanılmalıdır.



7. Geniřletilebilirlik

Flowly, yalnızca bařlangıçta sunulan özelliklerle sınırlı kalan kapalı bir uygulama deęildir. Sistem; yeni çalıřma biçimleri öğrenebilecek, farklı servislerle baęlantı kurabilecek ve ihtiyaç halinde yeni araçlar kazanabilecek şekilde tasarlanmıřtır. Böylece Flowly'nin ana yapısını deęiřtirmeden, kiřisel veya kurumsal ihtiyaçlara göre yeni yetenekler eklenebilir.

Flowly'yi geniřletmenin en kolay yolu Skill kullanmaktır. Skill'ler, belirli bir iřin nasıl yapılacaęını Agent'a öğreten tekrar kullanılabilir çalıřma tarifleridir. Örneęin řirketin raporlama yöntemi, bir projedeki kod inceleme kuralları veya müşteri taleplerinin deęerlendirilme süreci bir Skill hâline getirilebilir. Kullanıcı tamamlanan bir çalıřmayı /learn komutuyla yeni bir Skill olarak kaydedebilir. İstenirse Flowly, tekrar eden çalıřma biçimlerini fark ederek yeni Skill'ler oluřturabilir veya mevcut olanları geliřtirebilir. Bu özellik isteęe baęlıdır ve yapılan deęiřiklikler geri alınabilir.

Daha kapsamlı ihtiyaçlarda Plugin sistemi kullanılır. Plugin'ler Flowly'ye yeni araçlar, komutlar ve sistem davranıřları ekleyebilir. Örneęin řirkete özel bir veri kaynaęını sorgulayan araç, her görev sonunda otomatik kontrol yapan bir iřlem veya kullanıcı tarafından çağrılabilen özel bir komut geliřtirilebilir. Bu eklemeler Flowly'nin ana kodunu deęiřtirmeden yapılır. Kullanıcı tarafından yüklenen Plugin'ler açıkça etkinleřtirilmedikçe çalıřmaz ve hatalı bir Plugin, sistemin tamamı yerine yalnızca kendisinin devre dıřı kalmasına neden olur.

Flowly ayrıca MCP desteęi sayesinde GitHub, Notion, Linear, tarayıcı araçları veya kuruma ait özel servislerle baęlantı kurabilir. Baęlanan servislerin sunduęu iřlemler Agent tarafından normal bir araç gibi kullanılabilir. Hangi araçların kullanılabilceęi ayrıca sınırlandırılabilir. Flowly yalnızca MCP servislerini kullanmakla kalmaz; kendisi de bir MCP servisi olarak çalıřarak Claude Desktop, Cursor veya bařka Agent'lar tarafından kullanılabilir.

Geniřletilebilirlik yalnızca geliřtiricilere yönelik deęildir. Kullanıcılar, doęal dille ihtiyaçlarını anlatarak Flowlet adı verilen kalıcı ve etkileřimli küçük uygulamalar oluřturabilir. Takip ekranları, kontrol listeleri, formlar ve kiřisel panolar bunun örnekleridir. Böylece Flowly'ye yeni bir kullanım alanı kazandırmak için her zaman kod yazılması gerekmez.

Flowly'nin modeli de tek bir saęlayıcıya baęlı deęildir. Farklı yapay zekâ modelleri veya yerel olarak çalıřan modeller arasında geçiř yapılabilirken aynı hafıza, araçlar ve çalıřma biçimleri korunur. Sonuç olarak Flowly; yeni bir süreç öğretilerek, harici bir servis baęlanarak, kod tabanlı bir yetenek eklenerek veya yeni bir kullanıcı ekranı oluřturularak büyütülebilir.

8. Flowly'i Farklı Kılan Teknik Özellikler

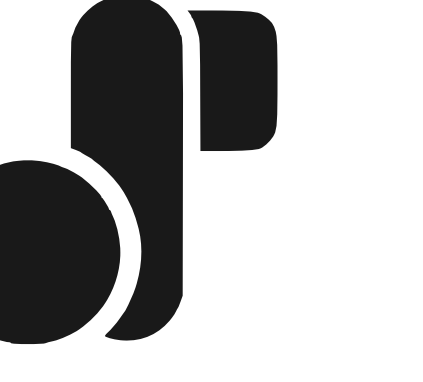
Flowly'yi farklı kılan temel özellik, yalnızca güçlü bir yapay zekâ modeli kullanması değildir. Flowly; model, hafıza, araçlar, görevler ve kullanıcı arayüzlerini aynı sistem içerisinde birleştirir. Bu nedenle yalnızca sorulara cevap veren bir sohbet uygulaması olarak değil, kullanıcıyla birlikte çalışan ve zaman içinde gelişen kalıcı bir Agent olarak hareket eder.

Yönetilen ve kalıcı hafıza: Flowly'nin hafızası yalnızca geçmiş konuşmaların saklanmasıyla oluşmaz. Kullanıcının tercihleri, verdiği kararlar, üzerinde çalıştığı projeler ve kişiler arasındaki ilişkiler ayrı yapılarda tutulur. Yeni bilgiler mevcut bilgilerle karşılaştırılır; tekrarlar birleştirilir, eski bilgiler güncelliğini yitirdiğinde geçmişe alınır ve kesin olmayan bilgiler kullanıcı onayına sunulur. Kullanıcı hatalı bir bilgiyi düzeltebilir veya hafızanın güvenilirliğine geri bildirim verebilir. Böylece hafıza zamanla büyüyen düzensiz bir not yığınınına dönüşmez.

Kanallar arasında aynı Agent: Terminal, masaüstü uygulaması, Telegram, Slack veya diğer bağlantılar birbirinden bağımsız botlar değildir. Kullanıcı farklı bir kanaldan Flowly'ye ulaştığında aynı hafıza, araçlar ve çalışma sistemi kullanılmaya devam eder. Örneğin telefondan oluşturulan bir görev masaüstünden takip edilebilir ve tamamlandığında sonuç görevin geldiği kanala gönderilebilir.

Sohbetin dışında çalışabilme: Flowly'nin çalışması kullanıcının mesajına cevap vermesiyle sona ermek zorunda değildir. Bir görev panoya kaydedilebilir, belirli bir zamanda çalıştırılabilir veya arka planda devam ettirilebilir. Büyük görevler daha küçük işlere ayrılarak birden fazla Agent tarafından paralel şekilde yürütülebilir. Her görevin durumu, sonucu ve karşılaşılan hatalar saklandığı için kullanıcı süreci sonradan inceleyebilir veya kaldığı yerden devam ettirebilir.

Kalıcı ve etkileşimli çıktılar: Flowly'nin ürettiği sonuçlar sohbet geçmişinde kaybolmak zorunda değildir. Raporlar, grafikler, kodlar, formlar ve diyagramlar sürümlü içerikler olarak saklanabilir ve kullanıcı paneline sabitlenebilir. Flowlet yapısı sayesinde kullanıcı doğal dille takip ekranları, kontrol listeleri, formlar ve küçük kişisel uygulamalar oluşturabilir. Flowlet içerisindeki günlük buton ve liste işlemleri her seferinde yapay zekâya sorulmadan, önceden belirlenmiş kurallarla çalışır. Bu sayede ekranlar hızlı ve tutarlı kalır.



8. Flowly'i Farklı Kılan Teknik Özellikler

Yerel ve modelden bağımsız yapı: Flowly'nin çalışma sistemi, hafızası, görevleri ve kalıcı çıktıları kullanıcının kendi cihazında tutulur. Yapay zekâ modeli ise sistemin değiştirilebilir bir parçasıdır. Kullanıcı farklı bir model sağlayıcısına veya kendi cihazında çalışan bir modele geçse bile Flowly'nin hafızası, araçları ve çalışma düzeni korunur. Başka bir ifadeyle model değişebilir; fakat kullanıcıyı tanıyan ve işleri yöneten sistem aynı kalır.

Flowly'nin teknik farkı şu cümleyle özetlenebilir:

Flowly yalnızca cevap üretmez; hatırlar, işleri sürdürür, farklı kanallarda kullanıcıyı takip eder ve ürettiği sonuçları kalıcı araçlara dönüştürür.

9. Ekler

Flowly ve geliştirici şirket Nocetic Limited ile ilgili resmî kaynaklara aşağıdaki bağlantılardan ulaşılabilir.

Flowly Ürün Bağlantıları

- Resmî web sitesi: useflowlyapp.com
- Ürün dokümantasyonu: useflowlyapp.com/en/docs
- Masaüstü uygulaması: useflowlyapp.com/en/download
- GitHub kaynak kodu: github.com/Nocetic/flowly
- Hata ve özellik talepleri: [GitHub Issues](https://github.com/Nocetic/flowly/issues)
- Python paketi: [PyPI – flowly-ai](https://pypi.org/project/flowly-ai/)
- iOS uygulaması: [Flowly – App Store](https://apps.apple.com/tr/app/flowly/id1544444444)
- Android uygulaması: [Flowly – Google Play](https://play.google.com/store/apps/details?id=com.nocetic.flowly)
- Flowly X hesabı: x.com/useflowlyai
-

Nocetic Limited

- Şirket web sitesi: nocetic.com
- LinkedIn: [linkedin.com/company/nocetic](https://www.linkedin.com/company/nocetic)
- GitHub: github.com/Nocetic
- X hesabı: x.com/nocetic
- İletişim: contact@nocetic.com

Yasal Belgeler

- Gizlilik Politikası: useflowlyapp.com/en/privacy
- Kullanım Koşulları: useflowlyapp.com/en/terms